

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Tecnología de Objetos				
TÍTULO DE LA PRÁCTICA:	Laboratorio 01				
NÚMERO DE PRÁCTICA:	1	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VI
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	MÍNIMO DE ESTUDIANTES	-----	MÁXIMO DE ESTUDIANTES	2
FECHA INICIO:	02/09/2026	FECHA FIN:	05/09/2026	DURACIÓN:	2 horas
INTEGRANTE (s): Chipana Jeronimo German Arturo Baca Calsin Leonardo Juan José					
DOCENTE(s): Edith Giovanna Cano Mamani					

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

CONTENIDO DE LA GUÍA

I. SOLUCIÓN

Repositorio:

<https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab1>

Scatie

<https://scastie.scala-lang.org/YTUgwDNWRcKgM9y4fOuS9g>

Instalación y configuración de Scala

Scala es un lenguaje de programación moderno que combina características de la programación orientada a objetos y la programación funcional. Se ejecuta sobre la **JVM (Java Virtual Machine)** y es utilizado para desarrollar aplicaciones, sistemas distribuidos y procesamiento de datos.

Para el desarrollo del curso se instaló **Scala CLI**, una herramienta que permite crear, ejecutar y administrar programas escritos en Scala desde la terminal.

Versión instalada:

- Scala CLI: **1.15.0**
- Scala: **3.8.4**
- Sistema operativo: **Windows**

Comandos utilizados

Para verificar la instalación de Scala CLI se utilizó:

```
None
scala-cli version
```

Para ejecutar un archivo de Scala:

None

```
scala-cli run vuelto.scala
```

También es posible ejecutar directamente:

None

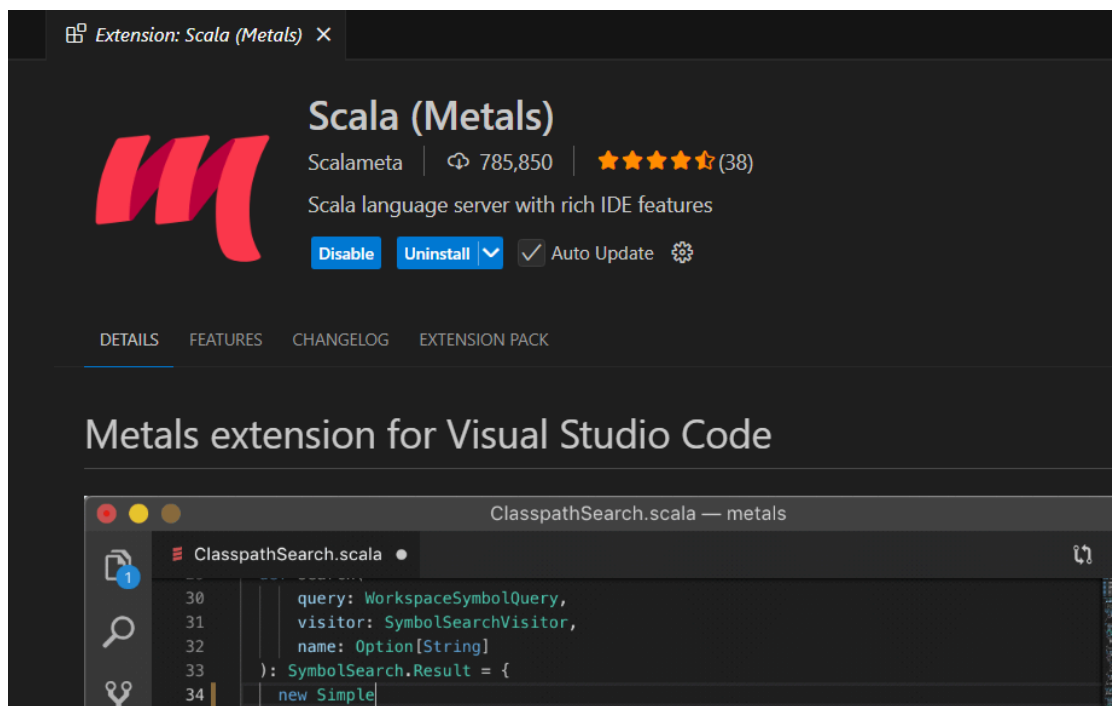
```
scala-cli vuelto.scala
```

Para actualizar Scala CLI se puede utilizar:

None

```
winget upgrade VirtusLab.ScalaCli
```

Finalmente, en **Visual Studio Code** se recomienda instalar la extensión **Scala (Metals)**, que proporciona funcionalidades como autocompletado, detección de errores, navegación del código y soporte para proyectos Scala.



Problema Propuesto:

Java

```
import scala.io.StdIn // Importamos la herramienta para leer del teclado

object Laboratorio01 {
  def main(args: Array[String]): Unit = {
    println("CALCULADORA DE VUELTO")

    // 1. Pedir la cantidad de dinero
    print("Ingresa la cantidad total (money): ")
    val money = StdIn.readInt()

    // 2. Pedir las denominaciones de monedas
    // El usuario debe ingresarlas separadas por espacios, ej: 1 2 5 10 20
    print("Ingresa las monedas disponibles separadas por espacio: ")
    val inputMonedas = StdIn.readLine()

    // Convertimos el texto ingresado en una Lista de Enteros
    val coins = inputMonedas.split(" ").map(_.toInt).toList

    // 3. Llamar a la función y mostrar resultado
    val soluciones = countChange(money, coins)

    println(s"\nPara un total de $money con monedas [{coins.mkString(", ")}]:")
    println(s"Hay $soluciones maneras diferentes de dar el vuelto.")
  }

  // LA FUNCIÓN RECURSIVA
  def countChange(money: Int, coins: List[Int]): Int = {
    if (money == 0) {
      1 // Encontramos una combinación exacta
    }
    else if (money < 0 || coins.isEmpty) {
      0 // No sirve: o nos pasamos o nos quedamos sin monedas
    }
    else {
      // 1. Usar la moneda actual (head): restamos su valor al dinero
      // 2. Ignorar la moneda actual y probar con el resto (tail)
      countChange(money - coins.head, coins) + countChange(money, coins.tail)
    }
  }
}
```

```
}  
}
```

Explicación Técnica de la Implementación

El desarrollo del laboratorio se divide en dos fases principales: la gestión de la entrada de datos por parte del usuario y la ejecución de la lógica algorítmica mediante recursión.

A. Interacción con el Usuario y Procesamiento de Datos

Para permitir que el programa sea dinámico, se utilizaron herramientas de la biblioteca estándar de Scala:

- **import scala.io.StdIn:** Se importa el módulo necesario para capturar datos desde la entrada estándar (teclado).
- **StdIn.readInt():** Esta función lee una línea de la consola y realiza un "parsing" para convertirla en un tipo de dato entero (Int), el cual representa la cantidad total de dinero a cambiar.
- **StdIn.readLine():** Captura una cadena de texto completa (String). Esto es necesario para recibir las denominaciones de las monedas en una sola línea.
- **Transformación de datos (.split(" ").map(_.toInt).toList):**
 - **.split(" "):** Divide la cadena de texto recibida cada vez que encuentra un espacio en blanco, generando un arreglo de caracteres.
 - **.map(_.toInt):** Es una función de orden superior que recorre cada elemento del arreglo y lo transforma de un tipo String a un tipo Int.
 - **.toList:** Convierte la colección resultante en una **Lista inmutable**, que es la estructura de datos requerida por la firma de la función countChange.

B. Análisis de la Lógica: Función countChange

La solución utiliza una estrategia de **Recursión Doble** (también conocida como ramificación), basada en el principio de que para cada tipo de moneda existen dos decisiones posibles:

1. Inclusión de la moneda (money - coins.head):

Se resta el valor de la moneda actual (la "cabeza" de la lista) al monto total. Se vuelve a llamar a la función con el nuevo monto reducido, pero manteniendo la lista de monedas completa. Esto permite que el algoritmo considere la posibilidad de utilizar la misma denominación múltiples veces (reuso).

2. Exclusión de la moneda (coins.tail):

Se llama a la función manteniendo el monto intacto, pero pasando únicamente el "resto" de la lista (todos los elementos excepto el primero). Esto obliga al algoritmo a buscar soluciones utilizando solamente las denominaciones restantes, descartando la moneda actual para esta rama de la ejecución.

Suma de Caminos:

La recursión termina sumando los resultados de ambas ramas. Cada vez que una rama llega al caso base de éxito (monto igual a 0), retorna un 1. La suma total de estos valores representa el número total de combinaciones únicas posibles para entregar el cambio.

Ejecución:

```
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab1> scala-cli run vuelto.scala
Compiling project (Scala 3.8.4, JVM (24))
Compiled project (Scala 3.8.4, JVM (24))
CALCULADORA DE VUELTO
Ingresa la cantidad total (money): 4
Ingresa las monedas disponibles separadas por espacio: 1 2

Para un total de 4 con monedas [1, 2]:
Hay 3 maneras diferentes de dar el vuelto.
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab1> scala-cli run vuelto.scala
CALCULADORA DE VUELTO
Ingresa la cantidad total (money): 10
Ingresa las monedas disponibles separadas por espacio: 1 2 5

Para un total de 10 con monedas [1, 2, 5]:
Hay 10 maneras diferentes de dar el vuelto.
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab1> scala-cli run vuelto.scala
CALCULADORA DE VUELTO
Ingresa la cantidad total (money): 10
Ingresa las monedas disponibles separadas por espacio: 2 5

Para un total de 10 con monedas [2, 5]:
Hay 2 maneras diferentes de dar el vuelto.
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab1> scala-cli run vuelto.scala
CALCULADORA DE VUELTO
Ingresa la cantidad total (money): 1000
Ingresa las monedas disponibles separadas por espacio: 1 2 5 10 20

Para un total de 1000 con monedas [1, 2, 5, 10, 20]:
Hay 22457476 maneras diferentes de dar el vuelto.
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab1> |
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

II. CUESTIONARIO

1. Explique el caso base implementado.

En esta implementación, la función `countChange` consolida el control de la recursividad en dos bloques lógicos que actúan como casos base:

- **El caso de éxito (`if (money == 0)`):** Se activa cuando las restas sucesivas del valor de las monedas igualan exactamente el monto objetivo. Al retornar 1, la pila de llamadas contabiliza que esta rama específica representa una combinación válida para el vuelto.
- **El caso de falla unificado (`else if (money < 0 || coins.isEmpty)`):** Agrupa matemáticamente dos escenarios de invalidez mediante un operador lógico OR. Por un lado, `money < 0` indica que la última moneda restada excedió la cantidad objetivo. Por otro lado, `coins.isEmpty` indica que la lista de denominaciones se agotó sin haber logrado reducir el dinero a cero. En cualquiera de estos subcasos, se retorna 0 para podar el árbol de recursión y descartar ese camino.

2. ¿Este problema se resuelve mejor en el modelo de programación funcional?

Sí. Aunque tu código incluye un bloque imperativo en el main para la interacción con el usuario mediante `StdIn`, la función principal `countChange` está modelada puramente bajo el paradigma funcional, lo cual es ideal por lo siguiente:

- **Ausencia de Estado Mutable:** No se utilizan variables contadoras locales o globales, ni bucles tradicionales (`for/while`). El estado fluye naturalmente a través de los parámetros en cada llamada recursiva.
- **Estructuras Inmutables:** El uso de las propiedades funcionales de la lista en Scala (`head` para el primer elemento y `tail` para el resto) permite dividir el problema de forma elegante sin necesidad de manejar índices, punteros, ni mutar la estructura de datos original.
- **Transparencia Referencial:** La función es matemáticamente pura. Para una misma entrada de `money` y `coins`, siempre producirá la misma salida y no genera efectos secundarios (*side-effects*), lo que hace que la lógica combinatoria sea predecible y fácilmente testeable.

3. ¿Qué beneficios se podrían obtener para este problema utilizando memoización?

La función implementada realiza dos llamadas recursivas por iteración (`countChange(money - coins.head, coins) + countChange(money, coins.tail)`). Esto genera un árbol de ejecución con subproblemas superpuestos (se recalculan combinaciones idénticas de dinero restante y lista de monedas múltiples veces). Aplicar memoización (cachear los resultados previamente calculados) otorgaría las siguientes ventajas de ingeniería:

- **Optimización de la Complejidad Temporal:** El tiempo de ejecución se reduciría de una escala exponencial a una pseudo-polinomial $O(M \cdot N)$ (donde M es el monto y N la cantidad de monedas). La función recuperaría valores del caché en tiempo constante $O(1)$ en lugar de expandir ramas redundantes.
- **Estabilidad ante Entradas Dinámicas:** Dado que el programa ahora permite ingresos arbitrarios del usuario vía teclado (`StdIn.readInt()`), valores de dinero altos y listas largas de monedas harían que la versión recursiva pura explote la memoria (*Stack Overflow*). La memoización mitigaría drásticamente la profundidad de la pila de llamadas, garantizando que el sistema escale y no colapse ante *inputs* grandes.

III. CONCLUSIONES Y RETROALIMENTACIÓN GENERAL

Conclusiones del Laboratorio

1. **Versatilidad del Lenguaje Scala:** Se concluye que Scala es un lenguaje sumamente potente para el manejo de lógica compleja. Su capacidad para integrar el modelo imperativo (entrada de datos vía `StdIn`) con el modelo funcional (procesamiento recursivo de listas) permite desarrollar aplicaciones robustas con menos líneas de código en comparación con lenguajes puramente imperativos como Java o C++.
2. **Eficacia de la Estrategia "Divide y Vencerás":** El problema del vuelto demostró que la recursión es una herramienta natural para resolver problemas combinatorios. Al dividir el problema en dos sub-problemas más simples (incluir o excluir una moneda), se logró una solución elegante que cubre todos los escenarios posibles sin necesidad de estructuras de control complejas.
3. **Importancia de los Casos Base:** El diseño de los casos base es el pilar de la programación recursiva. Se determinó que una correcta definición de las condiciones de salida (`money == 0` y `money < 0`) es lo que garantiza que el programa no solo retorne resultados correctos, sino que también finalice su ejecución de manera segura evitando ciclos infinitos.
4. **Límites de la Recursión Pura:** A través del análisis del problema, se concluye que aunque la recursión pura es semánticamente clara, tiene limitaciones de rendimiento ante entradas de gran volumen. Esto establece la base para estudiar técnicas de optimización avanzadas como la programación dinámica y la memoización en futuros laboratorios.

Retroalimentación General

- **Curva de Aprendizaje:** La sintaxis de Scala presenta un reto inicial, especialmente en el manejo de colecciones inmutables. Sin embargo, una vez comprendido el uso de métodos como `head` y `tail`, el procesamiento de datos se vuelve mucho más intuitivo que el uso de índices manuales en arreglos tradicionales.

- **Paradigma Funcional vs. Imperativo:** El laboratorio permitió contrastar ambos mundos. Mientras que la entrada de datos se siente más natural de forma imperativa, la lógica de negocio (el cálculo del vuelto) es significativamente más fácil de razonar y mantener bajo un esquema funcional.
- **Aplicación Práctica:** Este ejercicio no solo refuerza la sintaxis de un nuevo lenguaje, sino que introduce conceptos fundamentales de las ciencias de la computación, como la profundidad de la pila de llamadas (*call stack*) y la eficiencia algorítmica, los cuales son vitales para cualquier desarrollador de software profesional.
- **Entorno de Desarrollo:** El uso de Visual Studio Code junto con el ecosistema de Scala (Metals/Bloop) demostró ser un entorno de trabajo ligero y eficiente, facilitando la depuración y la ejecución rápida de scripts sin la sobrecarga de IDEs más pesados.